

Table of Contents

1) INTRODUCTION.....	2
2) TASK 1: THE LONG-RUNNING PROCESS.....	2
a) C Source Code.....	2
b) Terminal Execution & Verification.....	3
3) TASK 2: PROCESS IDENTITY (PID AND PPID).....	3
a) C Source Code.....	3
b) Terminal Execution & Verification.....	4
4) TASK 3: EXIT CODES AND OS FEEDBACK.....	4
a) C Source code.....	5
b) Terminal Execution & Verification.....	5
5) TASK 4: STANDARD I/O STREAMS.....	6
a) C Source Code.....	6
b) Terminal Execution.....	6
6) TASK 5: CONDITIONAL EXECUTION AND TERMINATION.....	7
a) C Source Code.....	7
b) Terminal Execution & Verification.....	8
7) CONCLUSION.....	9

List of figures

Listing 1. task1 alive.c.....	4
Listing 2. Terminal 1: Run in background.....	4
Listing 3. Terminal 2: Monitor the process.....	4
Listing 4. task2 identity.c.....	5
Listing 5. Terminal Commands.....	5
Listing 6. task3 exit.c.....	6
Listing 7. Checking Exit Status.....	7
Listing 8. task4 input.c.....	7
Listing 9. Running Task 4.....	8
Listing 10. task5 control.c.....	8
Listing 11. Testing Control Flow.....	9

Lab 3: Investigating Process Lifecycles and OS Interaction

1) INTRODUCTION

In Lecture 3, we learned that an OS manages processes and resources. In this lab, we bring that theory to life. Instead of just writing code that prints text, we will write code that interacts with the Linux kernel to query its own process identity, control its execution time, and report its success or failure back to the OS shell.

2) TASK 1: THE LONG-RUNNING PROCESS

OS Insight: In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

a) C Source Code

We create a program that runs for 30 seconds using a loop and sleep().



```
GNU nano 8.7.1 task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting... \n");

    // loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); //pauses execution for 1 second
    }

    printf("I am finished.\n");
    return 0;
}
```

Listing 1. task1 alive.c

b) Terminal Execution & Verification

To make this interesting, we run the program in the background using `&` so we can use a second terminal to inspect it while it's alive.

```
bipin@bipin-VirtualBox:~/batch38$ nano task1_alive.c
bipin@bipin-VirtualBox:~/batch38$ gcc task1_alive.c -o task1
bipin@bipin-VirtualBox:~/batch38$ ./task1 &
[1] 22985
I am starting...
bipin@bipin-VirtualBox:~/batch38$ I am finished.
```

Listing 2. Terminal 1: Run in background

```
bipin@bipin-VirtualBox: ~/batch38
bipin@bipin-VirtualBox:~/batch38$ ps aux | grep task1
bipin    22997  0.0  0.0 18000 2580 pts/0    S+   14:39   0:00 grep --color=auto task1
bipin@bipin-VirtualBox:~/batch38$
```

Listing 3. Terminal 2: Monitor the process

3) TASK 2: PROCESS IDENTITY (PID AND PPID)

OS Insight: Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually your terminal shell.

a) C Source Code

```
GNU nano 8.7.1 task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main () {
    //Get current process ID
    pid_t my_pid = getpid();

    //Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");

    return 0;
}
```

Listing 4: task2 identity.c

b) Terminal Execution & Verification

```
bipin@bipin-VirtualBox:~/batch38$ nano task2_identity.c
bipin@bipin-VirtualBox:~/batch38$ gcc task2_identity.c -o task2
bipin@bipin-VirtualBox:~/batch38$ ./task2 &
[1] 23041
My PID is: 23041
my Parent PID is: 23041
Sleeping for 20 seconds...
[1]+ Done ./task2
bipin@bipin-VirtualBox:~/batch38$ ps -p 23041 -o pid,ppid,cmd
PID PPID CMD
bipin@bipin-VirtualBox:~/batch38$
```

Listing 5. Terminal Commands

4) TASK 3: EXIT CODES AND OS FEEDBACK

OS Insight: When a program finishes, it returns an integer to the OS. 0 means "Success", and any non-zero number (like 1) means "Error/Failure". The shell stores this in \$?.

a) C Source code

```
GNU nano 8.7.1 task3_exit.c
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail):");
    scanf("%d", &num);
    if(num > 0) {
        printf("Success\n");
        return 0; // tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: error
    }
}
```

Listing 6. task3 exit.c

b) Terminal Execution & Verification

```
bipin@bipin-VirtualBox:~/batch38$ nano task3_exit.c
bipin@bipin-VirtualBox:~/batch38$ gcc task3_exit.c -o task3
bipin@bipin-VirtualBox:~/batch38$ ./task3
Enter a number (positive for success, negative for fail):5
Success
bipin@bipin-VirtualBox:~/batch38$ echo $?
0
bipin@bipin-VirtualBox:~/batch38$ ./task3
Enter a number (positive for success, negative for fail):-5
Failure
bipin@bipin-VirtualBox:~/batch38$ echo $?
1
bipin@bipin-VirtualBox:~/batch38$
```

Listing 7. Checking Exit Status

5) TASK 4: STANDARD I/O STREAMS

OS Insight: The OS provides standard input (stdin) and standard output (stdout) streams. scanf reads from stdin, and printf writes to stdout.

a) C Source Code

```
GNU nano 8.7.1 task4_input.c
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    //read string from standard input
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```

Listing 8. task4 input.c

b) Terminal Execution

```
bipin@bipin-VirtualBox:~/batch38$ nano task4_input.c
bipin@bipin-VirtualBox:~/batch38$ gcc task4_input.c -o task4
bipin@bipin-VirtualBox:~/batch38$ ./task4
Enter your name: bipin
Hello, bipin! Welcome to OS Class.
bipin@bipin-VirtualBox:~/batch38$
```

Listing 9. Running Task 4

6) TASK 5: CONDITIONAL EXECUTION AND TERMINATION

OS Insight: Programs often branch based on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.

a) C Source Code

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    //Print PID at start
    printf("Current Pid: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; //Success
    } else {
        printf("exiting...\n");
        return 1; //Failure/Abort
    }
}
```

Listing 10. task5 control.c

b) Terminal Execution & Verification

```
bipin@bipin-VirtualBox:~/batch38$ ./task5
Current Pid: 23393
Do you want to continue? (1 for Yes, 0 for No): 0
exiting...
bipin@bipin-VirtualBox:~/batch38$ echo $?
1
bipin@bipin-VirtualBox:~/batch38$ ./task5
Current Pid: 23401
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
^[[A^[[Abipin@bipin-VirtualBox:~/batch38$ echo $?
0
bipin@bipin-VirtualBox:~/batch38$ █
```

Listing 11. Testing Control Flow

7) CONCLUSION

This lab demonstrated that C programs do not exist in a vacuum; they are active processes managed by the Linux kernel. By utilizing system calls like `getpid()`, `sleep()`, and return codes, we successfully bridged the gap between high-level C code and low-level OS process management.

